

Panduan Pengguna

Akses GCP via Workload Identity

Federation

Multi-Language (Python, Node.js, Go, Java, Shell)
Menggunakan Microsoft Entra ID sebagai Identity Provider

Disusun oleh 00361920 — 2026

Daftar Isi

1. Pendahuluan
2. Arsitektur & Alur Autentikasi
3. Prasyarat
4. Network Requirements (Firewall/Proxy)
5. Konfigurasi
6. Implementasi
 - 6.1 Python
 - 6.2 Node.js
 - 6.3 Go
 - 6.4 Java
 - 6.5 Shell/gcloud
7. Verifikasi
8. Pemecahan Masalah
9. Pertanyaan Umum (FAQ)
10. Keamanan

1. Pendahuluan

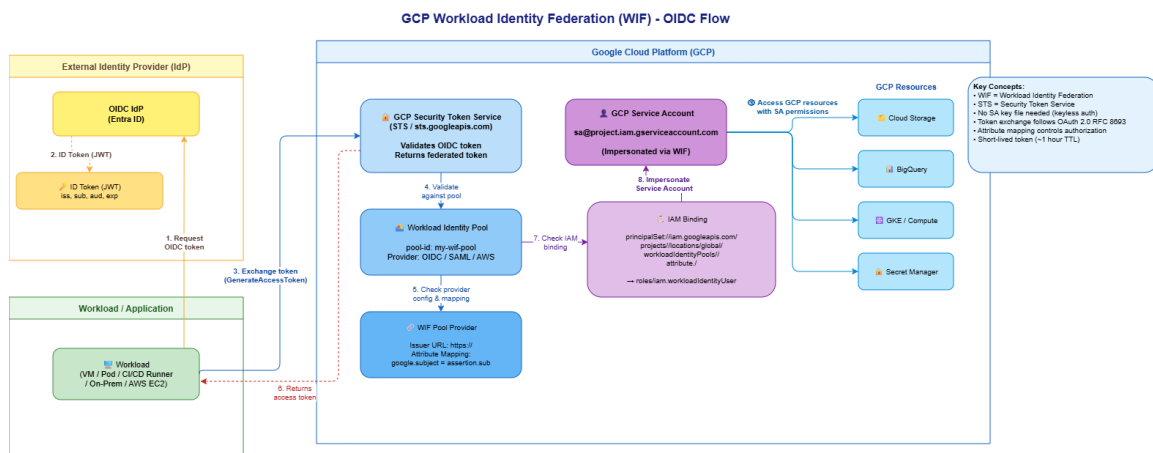
Dokumen ini merupakan panduan bagi pengguna yang perlu mengakses resource Google Cloud Platform (GCP) tanpa menggunakan Service Account Key permanen. Dengan mekanisme Workload Identity Federation (WIF), pengguna cukup menggunakan Client ID dan Client Secret yang disediakan oleh Tim SEA dari Microsoft Entra ID, dan sistem akan secara otomatis menukarkan token tersebut menjadi temporary GCP credentials.

Keuntungan

- Tidak ada Service Account Key permanen yang perlu dikelola
- Credentials bersifat sementara (otomatis kedaluwarsa setelah 1 jam)
- Jejak audit yang jelas di GCP Cloud Audit Logs
- Client Secret dapat dirotasi tanpa mengubah konfigurasi GCP
- Mendukung berbagai bahasa pemrograman

2. Arsitektur & Alur Autentikasi

Sistem ini menggunakan Workload Identity Federation untuk menukarkan token Entra ID menjadi GCP credentials melalui impersonasi Service Account:



(1) Aplikasi mengirim permintaan token ke Entra ID (client_credentials)

(2) Entra ID mengembalikan JWT access token

(3) Aplikasi mengirim token ke GCP STS (token exchange)

(4) GCP STS mengembalikan federated access token

(5) Aplikasi menggunakan federated token untuk impersonate Service Account

(6) GCP IAM mengembalikan Service Account access token

(7) Aplikasi menggunakan SA token untuk mengakses GCP resources

(8) GCP resources merespons sesuai permission Service Account

3. Prasyarat

Sebelum memulai, pastikan Anda memiliki:

No	Item	Keterangan
1	Runtime/SDK	Python 3.8+ / Node.js 18+ / Go 1.21+ / Java 11+
2	Informasi dari Tim SEA	Client ID, Tenant ID, Client Secret
3	GCP Project ID	Project tempat WIF dikonfigurasi
4	Workload Identity Pool ID	Pool ID yang sudah dibuat Tim SEA
5	Provider ID	Provider ID di dalam pool
6	Service Account Email	SA yang akan di-impersonate

PENTING: Seluruh informasi akan diberikan oleh Tim SEA. Jangan membagikan Client Secret kepada pihak lain.

4. Network Requirements (Firewall/Proxy)

Server atau aplikasi Anda WAJIB dapat mengakses domain berikut agar Workload Identity Federation berfungsi. Pastikan firewall, proxy, atau security group mengizinkan koneksi outbound HTTPS (port 443) ke domain-domain ini.

4.1 Microsoft Entra ID (Token Issuance)

Domain ini diperlukan untuk mendapatkan token dari Entra ID:

Domain	Port	Fungsi
login.microsoftonline.com	443	Token endpoint utama (v1 & v2)
login.microsoft.com	443	Alternative login endpoint
sts.windows.net	443	Security Token Service
login.windows.net	443	Legacy login endpoint

4.2 GCP Core APIs (Wajib)

Domain ini diperlukan untuk token exchange dan impersonation:

Domain	Port	Fungsi
sts.googleapis.com	443	Security Token Service - Token exchange
iamcredentials.googleapis.com	443	SA impersonation - generateAccessToken
iam.googleapis.com	443	IAM API - permission check
oauth2.googleapis.com	443	OAuth2 token operations

4.3 GCP Service APIs (Sesuai Kebutuhan)

Buka akses sesuai resource yang akan digunakan:

Domain	Port	Fungsi
storage.googleapis.com	443	Google Cloud Storage
bigquery.googleapis.com	443	BigQuery

compute.googleapis.com	443	Compute Engine
container.googleapis.com	443	Google Kubernetes Engine
pubsub.googleapis.com	443	Pub/Sub
logging.googleapis.com	443	Cloud Logging
monitoring.googleapis.com	443	Cloud Monitoring
secretmanager.googleapis.com	443	Secret Manager
cloudfunctions.googleapis.com	443	Cloud Functions
run.googleapis.com	443	Cloud Run

4.4 Wildcard Rules (Simplified)

Jika firewall mendukung wildcard, gunakan rules berikut:

```
*.microsoftonline.com:443
*.microsoft.com:443
*.windows.net:443
*.googleapis.com:443
```

4.5 Testing Konektivitas

Gunakan script berikut untuk memverifikasi konektivitas dari server Anda:

```
#!/bin/bash
# test_connectivity.sh - Test network connectivity for GCP WIF

DOMAINS=(
  "login.microsoftonline.com"
  "sts.windows.net"
  "sts.googleapis.com"
  "iamcredentials.googleapis.com"
  "iam.googleapis.com"
  "storage.googleapis.com"
)

echo "Testing network connectivity for GCP WIF..."
echo "====="
for domain in "${DOMAINS[@]}; do
  if curl -s --connect-timeout 5 -o /dev/null "https://$domain" 2>/dev/null; then
    echo "[OK]   $domain"
  else
    echo "[FAIL] $domain - Tidak dapat diakses"
  fi
done
```

PENTING: Apabila ada domain yang tidak dapat diakses, hubungi Tim Network/Infrastructure untuk membuka akses firewall ke domain tersebut sebelum melanjutkan.

5. Konfigurasi

Buat file `.env` atau atur environment variables:

```
# Microsoft Entra ID (dari Tim SEA)
AZURE_TENANT_ID=xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxxxx
AZURE_CLIENT_ID=xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxxxx
AZURE_CLIENT_SECRET=your-client-secret-value

# GCP Workload Identity Federation (dari Tim SEA)
GCP_PROJECT_NUMBER=123456789012
GCP_POOL_ID=my-wif-pool
GCP_PROVIDER_ID=entra-id-provider
GCP_SERVICE_ACCOUNT_EMAIL=my-sa@my-project.iam.gserviceaccount.com
```

PENTING: Jangan menyimpan file `.env` ke dalam Git repository.

6. Implementasi

6.1 Python

Instalasi dependensi:

```
pip install google-auth google-cloud-storage requests python-dotenv
```

Contoh script (access_gcp.py):

```
import os
import json
import requests
from dotenv import load_dotenv
from google.auth import impersonated_credentials
from google.auth.transport.requests import Request
from google.oauth2 import credentials as oauth2_credentials

load_dotenv()

# Step 1: Mendapatkan token dari Entra ID
token_url = (
    f"https://login.microsoftonline.com/"
    f"{os.getenv('AZURE_TENANT_ID')}/oauth2/v2.0/token"
)
token_resp = requests.post(token_url, data={
    "grant_type": "client_credentials",
    "client_id": os.getenv("AZURE_CLIENT_ID"),
    "client_secret": os.getenv("AZURE_CLIENT_SECRET"),
    "scope": f"{os.getenv('AZURE_CLIENT_ID')}/.default",
})
token_resp.raise_for_status()
entra_token = token_resp.json()["access_token"]
print("Step 1: Token Entra ID berhasil diperoleh")

# Step 2: Exchange token via GCP STS
project_number = os.getenv("GCP_PROJECT_NUMBER")
pool_id = os.getenv("GCP_POOL_ID")
provider_id = os.getenv("GCP_PROVIDER_ID")

sts_url = "https://sts.googleapis.com/v1/token"
audience = (
    f"https://iam.googleapis.com/projects/{project_number}"
    f"/locations/global/workloadIdentityPools/{pool_id}"
    f"/providers/{provider_id}"
)

sts_resp = requests.post(sts_url, json={
    "grant_type": "urn:ietf:params:oauth:grant-type:token-exchange",
    "audience": audience,
    "scope": "https://www.googleapis.com/auth/cloud-platform",
    "requested_token_type": "urn:ietf:params:oauth:token-type:access_token",
    "subject_token_type": "urn:ietf:params:oauth:token-type:jwt",
    "subject_token": entra_token,
})
sts_resp.raise_for_status()
federated_token = sts_resp.json()["access_token"]
print("Step 2: GCP STS token exchange berhasil")

# Step 3: Impersonate Service Account
```

```

sa_email = os.getenv("GCP_SERVICE_ACCOUNT_EMAIL")
impersonate_url = (
    f"https://iamcredentials.googleapis.com/v1/projects/-"
    f"/serviceAccounts/{sa_email}:generateAccessToken"
)
imp_resp = requests.post(impersonate_url,
    headers={"Authorization": f"Bearer {federated_token}"},
    json={
        "scope": ["https://www.googleapis.com/auth/cloud-platform"],
        "lifetime": "3600s",
    }
)
imp_resp.raise_for_status()
sa_token = imp_resp.json()["accessToken"]
print("Step 3: Impersonate Service Account berhasil")

# Step 4: Menggunakan SA token untuk akses GCP
# Contoh: List GCS buckets
gcs_resp = requests.get(
    "https://storage.googleapis.com/storage/v1/b",
    headers={"Authorization": f"Bearer {sa_token}"},
    params={"project": os.getenv("GCP_PROJECT_NUMBER")}
)
if gcs_resp.status_code == 200:
    buckets = gcs_resp.json().get("items", [])
    print(f"Step 4: Berhasil. Ditemukan {len(buckets)} bucket.")
    for b in buckets[:5]:
        print(f"    - {b['name']}")
else:
    print(f"Step 4: Error - {gcs_resp.status_code}: {gcs_resp.text}")

```

6.2 Node.js

Instalasi dependensi:

```
npm install axios dotenv
```

Contoh script (access_gcp.js):

```
require("dotenv").config();
const axios = require("axios");

async function main() {
  // Step 1: Mendapatkan token dari Entra ID
  const tokenUrl = `https://login.microsoftonline.com/`
    + `${process.env.AZURE_TENANT_ID}/oauth2/v2.0/token`;
  const tokenResp = await axios.post(tokenUrl, new URLSearchParams({
    grant_type: "client_credentials",
    client_id: process.env.AZURE_CLIENT_ID,
    client_secret: process.env.AZURE_CLIENT_SECRET,
    scope: `${process.env.AZURE_CLIENT_ID}/.default`,
  }));
  const entraToken = tokenResp.data.access_token;
  console.log("Step 1: Token Entra ID berhasil diperoleh");

  // Step 2: Exchange token via GCP STS
  const { GCP_PROJECT_NUMBER, GCP_POOL_ID, GCP_PROVIDER_ID } = process.env;
  const audience = `//iam.googleapis.com/projects/${GCP_PROJECT_NUMBER}`
    + `/locations/global/workloadIdentityPools/${GCP_POOL_ID}`
    + `/providers/${GCP_PROVIDER_ID}`;

  const stsResp = await axios.post("https://sts.googleapis.com/v1/token", {
    grant_type: "urn:ietf:params:oauth:grant-type:token-exchange",
    audience,
    scope: "https://www.googleapis.com/auth/cloud-platform",
    requested_token_type: "urn:ietf:params:oauth:token-type:access_token",
    subject_token_type: "urn:ietf:params:oauth:token-type:jwt",
    subject_token: entraToken,
  });
  const federatedToken = stsResp.data.access_token;
  console.log("Step 2: GCP STS token exchange berhasil");

  // Step 3: Impersonate Service Account
  const saEmail = process.env.GCP_SERVICE_ACCOUNT_EMAIL;
  const impResp = await axios.post(
    `https://iamcredentials.googleapis.com/v1/projects/-`
    + `/serviceAccounts/${saEmail}:generateAccessToken`,
    {
      scope: ["https://www.googleapis.com/auth/cloud-platform"], lifetime: "3600s",
      headers: { Authorization: `Bearer ${federatedToken}` }
    }
  );
  const saToken = impResp.data.accessToken;
  console.log("Step 3: Impersonate Service Account berhasil");

  // Step 4: Akses GCP resource
  const gcsResp = await axios.get("https://storage.googleapis.com/storage/v1/b", {
    headers: { Authorization: `Bearer ${saToken}` },
    params: { project: GCP_PROJECT_NUMBER },
  });
  console.log(`Step 4: Ditemukan ${gcsResp.data.items?.length || 0} bucket`);
}
```

```
main().catch(console.error);
```

6.3 Java

Maven dependency:

```
<dependency>
  <groupId>com.google.auth</groupId>
  <artifactId>google-auth-library-oauth2-http</artifactId>
  <version>1.23.0</version>
</dependency>
```

Alur implementasi Java sama: request token, STS exchange, impersonate SA. Gunakan ExternalAccountCredentials dari google-auth-library.

6.5 Shell / gcloud

Prasyarat: curl, jq, gcloud CLI (opsional)

Contoh script (access_gcp.sh):

```
#!/bin/bash
set -e
source .env

# Step 1: Mendapatkan token dari Entra ID
TOKEN_URL="https://login.microsoftonline.com/${AZURE_TENANT_ID}/oauth2/v2.0/token"
TOKEN_RESP=$(curl -s -X POST "$TOKEN_URL" \
  -d "grant_type=client_credentials" \
  -d "client_id=${AZURE_CLIENT_ID}" \
  -d "client_secret=${AZURE_CLIENT_SECRET}" \
  -d "scope=${AZURE_CLIENT_ID}/.default")
ENTRA_TOKEN=$(echo "$TOKEN_RESP" | jq -r .access_token)
echo "Step 1: Token Entra ID berhasil diperoleh"

# Step 2: Exchange token via GCP STS
AUDIENCE="//iam.googleapis.com/projects/${GCP_PROJECT_NUMBER}/locations/global/workloadId
entityPools/${GCP_POOL_ID}/providers/${GCP_PROVIDER_ID}"
STS_RESP=$(curl -s -X POST "https://sts.googleapis.com/v1/token" \
  -H "Content-Type: application/json" \
  -d "{
    \"grant_type\": \"urn:ietf:params:oauth:grant-type:token-exchange\",
    \"audience\": \"${AUDIENCE}\",
    \"scope\": \"https://www.googleapis.com/auth/cloud-platform\",
    \"requested_token_type\": \"urn:ietf:params:oauth:token-type:access_token\",
    \"subject_token_type\": \"urn:ietf:params:oauth:token-type:jwt\",
    \"subject_token\": \"${ENTRA_TOKEN}\"
  }")
FED_TOKEN=$(echo "$STS_RESP" | jq -r .access_token)
echo "Step 2: GCP STS token exchange berhasil"

# Step 3: Impersonate Service Account
IMP_RESP=$(curl -s -X POST \
  "https://iamcredentials.googleapis.com/v1/projects/-
/serviceAccounts/${GCP_SERVICE_ACCOUNT_EMAIL}:generateAccessToken" \
  -H "Authorization: Bearer ${FED_TOKEN}" \
  -H "Content-Type: application/json" \
  -d '{"scope":["https://www.googleapis.com/auth/cloud-platform"],"lifetime":"3600s"}')
SA_TOKEN=$(echo "$IMP_RESP" | jq -r .accessToken)
echo "Step 3: Impersonate Service Account berhasil"

# Step 4: Akses GCP resource (contoh: list buckets)
curl -s -H "Authorization: Bearer ${SA_TOKEN}" \
  "https://storage.googleapis.com/storage/v1/b?project=${GCP_PROJECT_NUMBER}" | jq
.items[].name
```

7. Verifikasi

Apabila seluruh langkah berhasil, output yang diharapkan:

```
Step 1: Token Entra ID berhasil diperoleh
Step 2: GCP STS token exchange berhasil
Step 3: Impersonate Service Account berhasil
Step 4: Berhasil. Ditemukan 3 bucket.
- bucket-data-prod
- bucket-logs
- bucket-backup
```

8. Pemecahan Masalah

Error	Penyebab	Solusi
Connection timeout	Firewall blok outbound port 443	Minta tim network whitelist domain (lihat Section 4)
SSL/TLS handshake failed	Proxy intercept SSL	Bypass proxy atau tambah CA certificate
DNS resolution failed	DNS tidak resolve public domain	Gunakan public DNS atau add DNS forwarder
invalid_client	Client ID atau Secret salah	Periksa nilai di .env
AADSTS700016	Client ID tidak ditemukan	Pastikan Tenant ID benar
INVALID_AUDIENCE (GCP STS)	Audience tidak sesuai dengan WIF pool/provider	Hubungi Tim SEA untuk verifikasi konfigurasi WIF
PERMISSION_DENIED pada generateAccessToken	Federated identity tidak diizinkan impersonate SA	Hubungi Tim SEA untuk cek IAM binding di Service Account
403 pada akses resource	Service Account tidak memiliki permission	Hubungi Tim SEA untuk menambahkan role/permission
Token expired	Token sudah kedaluwarsa	Jalankan ulang script (valid 1 jam)
INVALID_GRANT	Token Entra ID tidak valid untuk WIF	Pastikan scope dan audience sesuai konfigurasi Tim SEA

Apabila error tidak tercantum, hubungi Tim SEA dengan menyertakan:

- Screenshot pesan error lengkap
- Step mana yang gagal (1/2/3/4)
- Client ID yang digunakan (jangan kirimkan Client Secret)

9. Pertanyaan Umum (FAQ)

Q: Berapa lama credentials berlaku?

A: Token berlaku 1 jam. Setelah kedaluwarsa, jalankan ulang script.

Q: Apakah perlu menginstal gcloud CLI?

A: Untuk Python/Node.js/Go/Java: tidak. Untuk versi Shell: opsional (bisa pakai curl).

Q: Apa bedanya dengan Service Account Key?

A: WIF tidak memerlukan key file permanen. Credentials dihasilkan secara dinamis dan otomatis kedaluwarsa.

Q: Apakah dapat digunakan untuk CI/CD?

A: Ya. Simpan environment variables sebagai secret di CI/CD tool Anda.

Q: Resource GCP apa saja yang dapat diakses?

A: Bergantung pada role yang diberikan pada Service Account. Tanyakan kepada Tim SEA.

Q: Apakah mekanisme ini aman?

A: Ya. Tidak ada key permanen, credentials sementara, dan seluruh aktivitas tercatat di Cloud Audit Logs.

10. Keamanan

1. Jangan menyimpan credentials di source code

Selalu gunakan file .env atau environment variables.

2. Tambahkan .env ke .gitignore

Pastikan file .env tidak pernah ter-commit ke repository.

3. Jangan membagikan Client Secret

Jangan mengirimkan melalui email, chat, atau media tidak terenkripsi.

4. Laporkan apabila Client Secret bocor

Segera hubungi Tim SEA untuk revoke dan pembuatan secret baru.

5. Gunakan credentials sesuai kebutuhan

Jangan mengakses resource di luar cakupan yang ditentukan.

6. Jangan menyimpan token ke file

Token harus disimpan di memori saja.

Lampiran: Referensi Cepat

Item	Detail
Alur	Entra ID -> GCP STS -> Impersonate SA -> Resource
Python	pip install google-auth google-cloud-storage requests python-dotenv
Node.js	npm install axios dotenv
Go	cloud.google.com/go + golang.org/x/oauth2
Java	com.google.auth:google-auth-library-oauth2-http
Shell	curl + jq (gcloud opsional)
Masa berlaku	1 jam (otomatis kedaluwarsa)

-- Akhir Dokumen --